

Guía de migración a React — Scroll animado de fragancias

2026-09-21 · @Someone

Resumen ejecutivo

Este proyecto ("scroll") es una landing de fragancias con una experiencia de scroll paginado: cada fragancia ocupa una pantalla completa, con una botella 3D (three.js) que gira al cambiar de fragancia, un estallido de imágenes de ingredientes alrededor de la botella, un panel de texto (título + lista de ingredientes) y un fondo con crossfade.

Stack actual: HTML + CSS + JavaScript vanilla en ES Modules puros, sin bundler ni `package.json`. Todo se sirve como archivos estáticos (`python3 -m http.server` o Live Server). `three.js` está vendorizado a mano en `scripts/vendor/three/` (no viene de npm en tiempo de build). No hay ningún framework de UI: el DOM se arma y actualiza imperativamente desde `styles.js` / `motor.js`.

Objetivo de este documento: dar a quien tome el proyecto de migración a React un mapa completo de qué hace cada módulo, cómo se conectan, qué decisiones de diseño están detrás de cada comportamiento (para no perderlas en la reescritura) y qué partes del código/documentación vieja están obsoletas o nunca llegaron a aplicarse — para no confundir un diseño descartado con el comportamiento real.

Arquitectura de archivos (`scripts/`)

El código vive en `scripts/`, repartido en ES Modules con un grafo de imports **acíclico**:

```
constants.js ← content.js ← config.js ← helpers.js ← styles.js ← motor.js
```

`grow.js`, `panel.js` y `layout.js` son **módulos hoja** (sin imports propios), importados por los de arriba.

Archivo	Responsabilidad
<code>constants.js</code>	Sólo datos: rutas a imágenes de producto/ingrediente + un objeto <code>Fragancia<Nombre></code> por cada una de las 10 fragancias (<code>{ idx, nombre, fragancia, ingredientes }</code>). Mapeo cargado a mano por el usuario — no inventar ni extender sin pedido explícito.
<code>grow.js</code>	Fórmula de escala de "grow" (el efecto de zoom de los ítems) y su inversa (<code>growScaleFor / growForScale</code>).
<code>panel.js</code>	Panel de título + lista de ingredientes de TEXTO (no confundir con el estallido de imágenes).
<code>content.js</code>	Arma <code>CONFIG.content</code> a partir de <code>constants.js</code> (una "caja" por fragancia con producto + panel + estallido de ingredientes) y expone <code>relayContent</code> para <code>resize</code> .
<code>layout.js</code>	Único lugar que decide geometría: tamaño/posición del producto, radios válidos para ingredientes, tipografía. Ver sección dedicada.
<code>config.js</code>	Constantes globales (<code>CONFIG</code>), incluida <code>pageHeightFor</code> .
<code>helpers.js</code>	Utilidades, incluida <code>applyCssVariables</code> (fija variables CSS como <code>--page-height</code> , <code>--label-transition-ms</code> , tipografía fluida).
<code>styles.js</code>	Todo el DOM: crea/actualiza los elementos de cada ítem (producto, ingredientes, panel), maneja transiciones de entrada/salida, fondo, pooling de canvas. El módulo más grande del proyecto.
<code>motor.js</code>	Orquestación: scroll/paginación, <code>goToIndex</code> , construcción de cajas (<code>buildBoxes</code>), listeners de input, overlay "Show fragancias".
<code>bottle.js</code>	Botella 3D con <code>three.js</code> : carga del modelo GLB, materiales, spin, hover tilt, pooling de renderer.

Historia: hasta el 2026-09-01 todo vivía en un único `script.js` en la raíz (ya borrado). `constants.js` / `content.js` se separaron el 2026-09-03 para separar datos de lógica de armado.

Motor de scroll y paginación (`motor.js`) — estado vigente

Modelo actual (el que corre en producción): cada fragancia ocupa exactamente una pantalla (`vh`). Cambiar de fragancia es un "salto" instantáneo de scroll (`window.scrollTo`, sin animación de scroll nativo) disparado por `goToIndex(index, content)`, capturando `wheel/keydown/touch` con `preventDefault`. Este es el modelo descrito en `arquitectura_scripts` y es el que hay que tomar como referencia — **no** el de scroll continuo nativo (ver sección de features obsoletas).

Overlay "Show fragancias" (feature vigente, la última rediseñada, 13 rondas de iteración). El grupo de fragancias tiene dos formas:

- **Colapsado (default):** una única pantalla en el flujo normal del documento, fondo marrón fijo (`COLLAPSED_BOX_BACKGROUND_COLOR = '#c9a878'`), sólo botella + título (sin ingredientes), scroll de página normal y libre. Gobernado por `animationsEnabled = false`.
- **Desplegado ("Show fragancias"):** una pantalla por fragancia, paginación por `goToIndex`, scroll bloqueado de punta a punta, sin excepciones en los bordes. `animationsEnabled = true`.

Se pasa de desplegado a colapsado por el botón `×` o por seguir scrolleando más allá de la primera/última fragancia — ambos casos llaman a la misma función, `collapseToFreeScroll(extraScroll = 0)`.

Piezas clave:

- `setAnimationsEnabled(enabled)` — único punto de mutación del modo; reconstruye geometría/contenido y sincroniza clases CSS (`fake-scroll-mode`, `poc-layers-hidden`).
- `buildBoxes(containerSelector, content, collapsed)` — arma geometría y contenido según el modo. En colapsado crea UNA caja `overflow:hidden` sin animación; en desplegado, una caja por fragancia con su propio fondo.
- `pageHeightFor(boxCount, vh)` (`config.js`) y `applyCssVariables(config, boxCount)` (`helpers.js`) — `boxCount` es 1 en colapsado, `content.length` en desplegado.
- `atScrollEdge(direction)` — true si un paso más se saldría de la paginación; dispara `collapseToFreeScroll`.

- **Agrupación de gestos** (`GESTURE_GAP_MS = 250` , `noteInputGesture`): agrupa eventos de wheel/keydown separados por menos de ese silencio en un mismo id de gesto, para que una sola rueda de mouse (decenas de eventos) no atravesase varias fragancias de un tirón ni salga sola por el borde.
- **Orden de chequeos en los handlers**: `isLocked` se chequea ANTES que `atScrollEdge` — si hay una transición en curso, el intento de salida se encola (`queuedExitDirection` / `queueExitIfEdgeBound`) y se retoma recién al asentar la llegada, para no salir sin haber mostrado la caja de borde.

Por qué se llegó a este diseño (importante para no repetir errores en React): hubo un mecanismo previo ("modo fake", `isFakeScrollMode` / `enterFakeScrollMode` / `exitFakeScrollMode`) que liberaba el scroll nativo en los bordes de la paginación MIENTRAS el grupo seguía desplegado (11 pantallas de alto). Eso obligaba a convivir con dos físicas de scroll a la vez y costó 6 rondas de parches (casos de borde: caja "pelada" a mitad de cruce, scrolls perdidos durante el lock, cajas de borde salteadas, cruces de 7-8 scrolls, salidas disparadas por coletazos del mismo gesto). La solución de fondo fue **colapsar el grupo entero a una sola pantalla** al salir — así de ahí en más es scroll de página normal, sin estado intermedio. **Moraleja para React**: modelar el grupo de fragancias como una unidad que tiene un tamaño (1 pantalla colapsado, N pantallas desplegado) en vez de intentar que convivan dos modos de scroll sobre el mismo contenedor.

Sistema de layout/geometría (`layout.js`)

`layout.js` es el **único lugar que decide geometría** (tamaño, posición, tipografía). `layoutFor(vw, vh)` devuelve todo: modo, centro, semiejes del "stage", tamaño del producto, posiciones del panel, tamaños de fuente y topes de radio para ingredientes.

Dos modos por orientación (`vw >= vh`), no por breakpoint de ancho:

- `stack` (pantalla parada, mobile típico): panel en dos bandas (título arriba, lista abajo).
- `column` (pantalla acostada, desktop típico): misma estructura, sólo cambian los números (sufijo `_STACK` / `_COLUMN`).

Invariantes garantizados por construcción (no por chequeo probabilístico):

- El stage está SIEMPRE centrado en `(vw/2, vh/2)` .
- Las bandas de texto quedan FUERA del stage en los dos modos — el texto nunca tapa el producto/ingredientes.
- `productSize` se acota al stage: en pantallas muy bajas la botella se achica sola.

- Los ingredientes nunca tapan la botella (`radiusRangeAt` calcula el rango de radio válido ANTES de sortear, sin loop de reintentos) ni se pasan del borde superior/inferior del producto.

Constantes de tamaño que el usuario ajusta directamente (útil saberlas para exponerlas como props/config en React):

```
PRODUCT_VIEWPORT_HEIGHT_FRACTION_STACK = 0.66 // mobile
PRODUCT_VIEWPORT_HEIGHT_FRACTION_COLUMN = 2 // desktop, siempre clampea al máximo del stage
```

Son la fracción del alto del viewport que debe medir la botella VISIBLE. En desktop, el clamp contra el stage hace que subir la constante no tenga efecto — la botella ya ocupa el 100% del alto disponible. En mobile sí hay margen real, pero el clamp golpea en pantallas muy bajas (ej. iPhone SE).

`PRODUCT_CONTENT_X / PRODUCT_CONTENT_Y` (0.378 / 0.984) son la fracción de ancho/alto del cuadro cuadrado que ocupa el contenido REAL (no transparente) de la botella — medidas contra el alfa real del PNG viejo de producto, y reusadas hoy para calibrar el tubo placeholder de three.js y (indirectamente) el modelo GLB.

`INGREDIENT_CONTENT` (0.999) es el equivalente para ingredientes — cuánto del cuadro de cada imagen es contenido real tras el recorte al bounding box del canal alfa.

Constantes de tamaño de ingrediente en desktop (estado post-reversión, base para cualquier pedido futuro de "más grande/más chico"):

```
INGREDIENT_SIZE_MIN_FRACTION_COLUMN=0.13 ,
INGREDIENT_SIZE_MAX_FRACTION_COLUMN=0.22 , INGREDIENT_MAX_VS_PRODUCT=0.34 .
```

Modelo de radio válido por ángulo: para un ángulo dado, calcula `rMin` (fuera del núcleo del producto) y `rMax` (el ítem entero adentro del stage), y sortea el radio dentro de `[rMin, rMax]`. Sin loop de reintentos ni empuje correctivo — garantiza a la vez no-solapamiento y no-salirse-de-pantalla.

Para React: `layoutFor(vw, vh)` es prácticamente un hook puro (`useLayout(vw, vh)` con `useMemo / useLayoutEffect` sobre `resize`) — es la función más reutilizable del proyecto tal cual está, con cero dependencias del DOM.

Botella 3D con three.js (`bottle.js`)

Modelo: la botella es un modelo GLB real (`assets/models/bottle.glb` , 2MB), cargado con `GLTFLoader` (vendedorizado en `scripts/vendor/three/examples/jsm/`). Se abandonó una versión anterior por código (`LatheGeometry` con perfiles medidos a mano) el 2026-

09-09. `three.js` en sí está vendorizado (`three.module.js` + `three.core.js`, r185) sin bundler ni npm en el proyecto — para React hay que decidir si se sigue vendorizando o se instala como dependencia normal de npm.

Carga única, no por caja: `getMasterModel()` cachea una Promise a nivel de módulo — la primera caja dispara la carga, arma materiales y mide la caja (`Box3`); todas las cajas siguientes reusan ese resultado con `model.clone(true)` (clona jerarquía, comparte geometrías/materiales/texturas por referencia). `dispose()` sólo libera el renderer/contexto WebGL de esa caja puntual, nunca las geometrías/materiales/texturas compartidas.

Materiales (con `transmission` real, no `opacity` — clave para el aspecto "vidrio ámbar profundo"):

- Cuerpo: `MeshPhysicalMaterial`, `transmission: 1`, `thickness: 0`, `attenuationColor / attenuationDistance` (Beer-Lambert) para el tinte, `clearcoat 0.5`, `ior 1.5`, `side: FrontSide`.
- `innerWall`: copia de la misma geometría del cuerpo, material opaco en `BackSide` — sin esto la transmisión lee el canvas transparente y el vidrio se ve vacío/fino.
- Tapa/pico: `MeshStandardMaterial` metal dorado, `metalness 1`.
- Tubo interior: `MeshStandardMaterial` opaco oscuro — visible en silueta a través del vidrio.
- Etiqueta: cáscara curva (no un plano) con textura PNG, `alphaTest 0.4`, `side: DoubleSide`.

Etiqueta dinámica por fragancia (`LABEL_URLS_BY_SLUG`): 9 de las 10 fragancias tienen su propio PNG de etiqueta (Painkiller usa la default). Se resuelve con `resolveLabelUrl(fragranceSlug)`. La etiqueta se arma por CLON en `buildBottleRig`, no en el modelo maestro compartido — dos botellas de la misma fragancia reusan la misma textura ya cargada (material cacheado por URL en un `Map`).

Spin/rotación al cambiar de caja (ver también sección de Transiciones): `runSpin()` anima con easing partido (`EASINGS.in / .out`) para que las dos mitades (saliente/entrante) empalmen sin frenazo.

Hover tilt: mientras el mouse está sobre el canvas, flecha ↓/↑ inclina la botella ~5° (`HOVER_TILT_ANGLE_DEG`, animación 300ms `easeInOutCubic`). Bloquea la navegación de flechas en `motor.js` mientras el mouse está encima (`isBottleHovered()`).

Pooling de renderer (fix de performance, 2026-09-14): `createBottle` cachea `{renderer, envTarget}` por `<canvas>` en un `WeakMap` (`canvasRendererCache`) y los reusa entre cajas en vez de crear un `WebGLRenderer / PMREMGGenerator` /recompilar shaders de transmission desde cero cada vez — esto era la causa real de un delay de 2-3s

al scrollear (ver sección de Rendimiento). `dispose({ keepAlive })` no destruye el `renderer` si `keepAlive` es `true`. Hay además un pool de `<canvas>` del producto en `styles.js` (`acquireProductCanvas` / `releaseProductCanvas`, tope 2 — saliente + `prewarm` entrante).

Pendientes no implementados (anotados por si se retoman durante la migración): modelos GLB específicos por fragancia (hoy es un único modelo compartido con etiqueta variable), variante anisotrópica del dorado, bloom sutil, compartir un solo `renderer/PMREM` entre TODAS las cajas (hoy es por-canvas con pool de 2).

Transiciones entre cajas

Al cambiar de fragancia (`goToIndex` en `motor.js`) ocurren varias cosas EN PARALELO, coreografiadas con cuidado tras 5 rondas de ajuste el 2026-09-10:

1. **Giro continuo de la botella:** la botella saliente gira de frente a atrás; la entrante aparece YA de atrás (sin fade-in) y gira lo que le falta hasta el frente — sensación de una sola botella que nunca frena. Las dos mitades duran lo mismo (`COLLAPSE_TO_CENTER_MS = 500ms`) y usan la misma curva `easeInOutCubic` partida exactamente por la mitad (`easeInCubic` para la salida, `easeOutCubic` para la entrada) — así la velocidad angular coincide en el empalme y no hay frenazo-y-arranque.
2. **Collapse de ingredientes al centro:** los ingredientes de imagen (no el producto, no el panel) se trasladan hacia el centro de la botella + fade a opacity 0, en paralelo con el giro, durante los mismos 500ms (`collapseIngredientsToCenter`, `styles.js`).
3. **Crossfade de fondo** (`transitionBackground`, `styles.js`): una capa fija independiente del scroll (`.background-transition-layer`) con sólo DOS nodos que se turnan de rol entrante/saliente. Es un crossfade PURO de opacity — se probó parallax con desplazamiento (`translateY`) pero se sacó porque "se sentía como un salto" entre páginas; hoy sólo anima `opacity`, nunca `transform`.
4. **Prewarm del producto entrante:** `prewarmIncomingProduct(display)` arma el canvas+botella de la caja DESTINO oculto (`visibility: hidden`) y compila shaders/PMREM ANTES del salto — el costo caro de `createBottle` se paga durante el collapse, no en el instante del corte. `transitionDisplay` reusa ese canvas por identidad en vez de crear uno nuevo.
5. **Reveal de ingredientes entrantes:** llegan a su posición final EN EL MISMO instante en que aparece la caja nueva (no esperan a que la botella termine de girar) — se sacó a propósito el mecanismo viejo de "esperar al 80% del giro" tras pedido explícito del usuario.

Todas estas piezas se disparan síncronamente al arrancar `goToIndex` (fondo + collapse + prewarm), y sólo el swap de contenido real (`transitionDisplay`) espera el `setTimeout(COLLAPSE_TO_CENTER_MS)` completo.

Constante central: `COLLAPSE_TO_CENTER_MS` (`styles.js`) = 500ms gobierna CINCO cosas a la vez: las dos mitades del giro, el collapse de ingredientes salientes, la ventana del prewarm, y la traslación de ingredientes entrantes a su posición final. Subir/bajar este único número cambia la velocidad de todo a la vez, a propósito.

Carga inicial / resize (`setDisplayInstant`) usa un mecanismo distinto y deliberadamente NO tocado en la última ronda: los ingredientes entran ocultos hasta que el spin de la botella llega a `SPIN_REVEAL_AT_MS = 1600ms` de un giro clásico de 2s (`SPIN_REVEAL_FRACTION = 0.8` en versiones previas). Diverge a propósito de `transitionDisplay` porque el pedido del usuario fue específicamente sobre "cambiar de página", no sobre la carga inicial.

Sistema de ingredientes

Estallido normal ("explosión de imágenes"): para cada fragancia, `content.js` arma `imageSlots` eligiendo SIN repetición hasta `IMAGES_PER_INGREDIENT = 3` imágenes de cada ingrediente (`pickUniqueImages`) — un slot por IMAGEN elegida, no por ingrediente. Con 3 ingredientes × 3 imágenes salen 9 ítems de display por caja. Los ángulos del estallido se reparten sobre `imageSlots.length`. `layout.js` calcula, para cada ángulo, el rango de radio válido (`radiusRangeAt`) que evita solapar el producto y no salirse del stage — sin loop de reintentos.

Mecanismo de imagen fija (`testFixed`): para ciertas fragancias, en vez del estallido de 9 imágenes se muestra UNA sola imagen fija, sin traslación (entra/sale con fade + zoom, no con movimiento), sizeada al alto real de la botella y con ancho proporcional (nunca deformada). Nació como una "caja de prueba" (11ª caja, al final del scroll, con `almendrasdistribuidas.png`) y se generalizó el 2026-09-10 a fragancias reales vía `FIXED_INGREDIENT_IMAGE_OVERRIDES` (objeto `{ nombreFragancia: rutaAsset }` en `content.js`) — hoy activo para **Painkiller** y **Forbidden Flower**. Para agregar/sacar una fragancia de este mecanismo sólo hay que tocar ese objeto — el resto del pipeline (collapse, reveal, sizing) ya está condicionado por la propiedad `item.testFixed` del ítem, no por identidad de caja.

`applyAlmondTestLayout` (nombre heredado de la caja de prueba original, reusada tal cual para las fragancias reales) fija `item.size = altura real de la botella en px` (viene de `layout.product.halfH`) y deja `xi===xf`, `yi===yf` (sin traslación — `finalOffset` da `{dx:0, dy:0}`). En `styles.js`, `createImageElement` se ramifica por `item.testFixed`: un ingrediente normal fuerza `width===height` (cuadrado); un ítem `testFixed` fija sólo

`height` y deja `width: auto` para no deformar, y anula los `max-width / max-height` de `.display-image` (pensados para ingredientes normales, muy por debajo del alto real de la botella).

Para React: el patrón `item.testFixed` sugiere modelar el "contenido de ingredientes" de una caja como una unión discriminada — `{ kind: 'burst', slots: [...] }` vs `{ kind: 'fixed', image }` — en vez de una lista homogénea con un flag opcional.

Otros detalles relevantes

Tipografía: `.display-label` (clase base de título e ingredientes de texto) usa `font-family: 'Cormorant Garamond', serif` + `text-shadow` en dos capas (blur 8px/55% y blur 2px/65%, ambas negras) para que el texto blanco contraste sobre fondo/imagen. Los tamaños vienen de las CSS vars `--title-font-size / --ingredient-font-size`, calculadas en `layout.js`.

Ancla de prueba a Google en la botella (`ancla_google_botella`, pedido explícito de prueba, no un link real de producto): el canvas de la botella va envuelto en un `<a href="https://www.google.com/?fragrance=<slug>">` (`createProductLinkedCanvas`, `bottleLinkHref` en `styles.js`). Al migrar, decidir si este link de prueba se reemplaza por la URL real del producto o se retira — `bottleLinkHref` es el único lugar a tocar.

Secciones fake / demo: 3 secciones de referencia (`fake-section-before-1/2` antes del grupo, `fake-section-after` después) para probar la paginación dentro de una página real (fondo de color + título grande, sin relación con el contenido de producción). `boxTops` en `motor.js` calcula el offset real del contenedor (`containerTop`) porque estas secciones lo empujan hacia abajo. Al migrar, estas secciones probablemente no formen parte del componente reusable — son andamiaje de prueba del POC, no parte del producto final. Ver también la sección de Motor de scroll: la interacción entre estas secciones y el overlay "Show fragancias" fue la que forzó el rediseño a `collapseToFreeScroll`.

Rendimiento WebGL — lección importante para React

El usuario reportó un delay de 2-3s al scrollear entre fragancias. **No** era la lógica de posicionamiento de ingredientes (trigonometría barata, corre una sola vez por carga/resize). La causa real: `createBottle()` (`bottle.js`) creaba un `WebGLRenderer` nuevo + cocinaba el environment PMREM + compilaba los shaders del vidrio (`transmission` real, notoriamente caro) **desde cero en cada cambio de caja** — a pesar

de que los materiales en sí ya eran compartidos por referencia entre botellas, un contexto WebGL nuevo obliga a recompilar sus shaders igual (el caché de programas compilados de three.js vive por renderer/contexto, no se comparte entre contextos distintos).

Fix: pooling de renderer por `<canvas>` (`WeakMap`) + pool de canvases del producto (tope 2: saliente + prewarm entrante) — ver detalle en la sección de la botella 3D.

Para React: si se recrea este patrón con un componente `<Bottle>` por caja que se monta/desmonta en cada cambio de fragancia, hay que reproducir este pooling explícitamente (por ejemplo, mantener 1-2 renderers vivos en un ref compartido fuera del ciclo de montaje/desmontaje de React, o usar un único componente `<Bottle>` persistente que sólo cambia de fragancia/etiqueta en vez de desmontarse) — de lo contrario se reintroduce el delay de 2-3s que costó una sesión completa de diagnóstico resolver.

Gotchas de desarrollo (útiles al verificar la migración)

Caché de módulos ES: servir con `python3 -m http.server` (o Live Server) sin headers `Cache-Control` hace que Chrome sirva módulos ES viejos cacheados tras renombrar/agregar un export. Síntoma característico: se rompe TODO de golpe (0 cajas, 0 items, scroll muerto) porque un error de linkeo de módulo detiene toda la ejecución — no es un bug de lógica. Se confirma en consola: `Uncaught SyntaxError: The requested module './X.js' does not provide an export named 'Y'`. Con un bundler moderno (Vite, webpack, Next.js) para React este problema desaparece solo, pero vale la pena saber que existía para no perseguir un fantasma si algo similar reaparece durante la transición.

No confiar en la respuesta de éxito de una escritura remota de archivo (aplica si el flujo de deploy es similar) sin verificar el contenido real después: hubo casos donde una escritura de archivo remota reportó éxito (mtime actualizado) sin que el contenido nuevo llegara a disco. Moraleja general para cualquier pipeline de deploy: verificar el contenido servido después de un cambio importante, no confiar sólo en el código de respuesta de la operación de escritura.

⚠ Features descartadas o nunca aplicadas realmente

Muy importante para no confundir documentación vieja con el comportamiento real del código:

"Scroll continuo nativo" — NUNCA llegó a aplicarse al código real. Existe documentación detallada (y verificada con Playwright) de una reescritura de `motor.js` a scroll nativo continuo (sin `goToIndex` /paginación por salto, sincronizado por `scrollY`).

Esa reescritura se probó y validó en un sandbox de nube separado, pero **nunca se aplicó al archivo real del proyecto** — se confirmó leyendo el `motor.js` real y el historial de git, que siguen con el modelo de "salto de página" de siempre. **El modelo vigente y real es el descrito en la sección "Motor de scroll y paginación" de este documento** (`goToIndex`, `isLocked`, `LOCK_MS`, colapsado/desplegado). Si en algún momento se quiere scroll continuo real, hay que implementarlo desde cero sobre el código actual — el diseño viejo puede servir de referencia conceptual, pero no como parche para aplicar sin revisar.

Swipe táctil manual — obsoleto, reemplazado por scroll nativo. Hubo una implementación de swipe táctil a mano (`touchstart/move/end/cancel` en `motor.js`, con `touch-action: none` global) para resolver el bloqueante de mobile. Quedó completamente eliminada el 2026-09-08 cuando (según la documentación) se migró a scroll nativo — pero dado el punto anterior, conviene **verificar en el código real actual si el swipe táctil manual sigue vigente o no** antes de asumir cuál de los dos modelos gobierna hoy el touch. El método de testing (Playwright + CDP `Input.dispatchTouchEvent`) sigue siendo válido para probar gestos táctiles reales si hace falta reproducir el comportamiento en React.

Recomendación operativa: antes de empezar la migración, correr el sitio real (el servidor del usuario) y confirmar en vivo cuál de estos dos modelos de scroll/touch está efectivamente activo, en vez de guiarse sólo por esta documentación histórica.

Recomendaciones para la migración a React

Maapeo sugerido módulo → React

Módulo actual	Equivalente sugerido en React
<code>constants.js</code>	Datos estáticos (JSON/TS) — sin cambios de fondo
<code>layout.js</code>	Hook puro <code>useLayout(vw, vh)</code> (ya es prácticamente una función pura sin DOM)
<code>grow.js</code>	Utilidad pura, sin cambios
<code>bottle.js</code>	Componente <code><Bottle></code> con <code>three.js</code> / <code>@react-three/fiber</code> , cuidando reproducir el pooling de renderer (ver sección de Rendimiento)
<code>content.js</code> / <code>panel.js</code>	Lógica de armado de datos por fragancia — puede vivir como funciones puras que alimentan el estado de React, no como generador de DOM

Módulo actual	Equivalente sugerido en React
<code>styles.js</code>	Se disuelve casi entero: pasa a ser JSX + CSS/CSS-in-JS + <code>useState</code> / <code>useRef</code> por cada tipo de ítem (producto, ingrediente-estallido, ingrediente-fijo, panel)
<code>motor.js</code>	Un hook de orquestación (<code>useFragranceScroll</code> o similar) que maneje <code>currentIndex</code> , <code>animationsEnabled</code> (colapsado/desplegado) y los listeners de scroll/wheel/keydown

Estado central a modelar explícitamente (hoy disperso en variables de módulo de `motor.js` / `styles.js`):

- `currentIndex` (fragancia actual)
- `animationsEnabled` (colapsado vs desplegado)
- `isLocked` / transición en curso
- `pendingIncomingBottle` / prewarm (si se decide mantener el prewarm como optimización)

Puntos de mayor riesgo/complejidad al migrar (donde vale la pena invertir tiempo de diseño antes de escribir código):

1. **La coreografía de transición** (giro + collapse + fondo + prewarm, todos sincronizados por el mismo `COLLAPSE_TO_CENTER_MS`) — es el corazón de la experiencia y el que más rondas de ajuste llevó. Recomendado: modelarlo como una única máquina de estados o timeline (ej. con Framer Motion `AnimatePresence` / `useAnimate`, o GSAP timeline) en vez de varios `setTimeout` coordinados a mano.
2. **El pooling de renderer WebGL** — crítico para performance, fácil de perder si `<Bottle>` se monta/desmonta por fragancia en vez de persistir.
3. **El modelo colapsado/desplegado del overlay** — evitar el error original de mezclar dos físicas de scroll; modelar el grupo como una unidad con un tamaño variable (1 pantalla vs N pantallas), no como un contenedor fijo con excepciones en los bordes.
4. **Verificar primero si el modelo real de scroll/touch es el de "salto de página" o si alguna sesión posterior sí aplicó el scroll continuo** (ver sección de Features obsoletas) — esto cambia la estrategia de migración de raíz.

Sugerencia de orden de migración: (1) `layout.js` como hook puro (bajo riesgo, alto reuso); (2) render estático de una fragancia (producto + ingredientes + panel, sin transiciones) para validar el layout en React; (3) botella 3D con pooling; (4) transiciones de caja a caja; (5) overlay colapsado/desplegado y scroll/gestos.